

APPENDIX A

SUPPORTING R CODE

This chapter contains the R code that will help students in using this text for the course. To use basic graphics, generate data and perform statistical analysis based on resampling methods. The information in this chapter is organized according to when it might be needed first.

Chapter 3.

```
factorial(n) # n!  
choose(n, k) #combinations  
factorial(n)/factorial(n-k) #permutations
```

Chapter 8.

Basic distributions and related graphs. Built in distribution functions: density or probability, cdf, quantiles can be found in Tables A.1–A.3.

■ EXAMPLE A.1

The code below shows how to graph three density functions in one figure, using solid, dotted, and dashed lines, respectively (see Figure A.1). The distributions used (BETA(2, 5), BETA(3, 3), BETA(1, 4)) can easily be substituted by other distributions from Table A.1. Ranges on both axes (xlim and ylim) will need to be adjusted accordingly (Figure A.1).

Table A.1 Built in distributions

Distribution	R function	Parameters	Formula
Binomial	binom	n, p	$\binom{n}{x} p^x (1-p)^{n-x}$
Poisson	pois	lambda	$p(x) = e^{-\lambda} \frac{\lambda^x}{x!}$
Uniform	unif	a, b	$f(x) = 1/(b-a), a \leq x \leq b$
Normal	norm	μ, σ	$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$
Exponential	exp	β	$f(x) = \beta^{-1} e^{-x/\beta}$
Gamma	gamma	α, β	$f(x) = [\beta^{-\alpha}/\Gamma(\alpha)] x^{\alpha-1} e^{-x/\beta}$
Chi-square	chisq	ν	$f(x) = \frac{1}{\Gamma(\nu/2) 2^{\nu/2}} x^{\nu/2-1} e^{-x/2}$
Student's t	t	ν	$\frac{\Gamma((\nu+1)/2)}{\sqrt{\nu\pi}\Gamma(\nu/2)} (1+x^2/\nu)^{-(\nu+1)/2}$
Weibull	weibull	α, β	$f(x) = \alpha\beta x^{\alpha-1} e^{-\beta x^\alpha}$
Beta	beta	α, β	$f(x) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1}$

Table A.2 Densities/pdf's and cdf's of built in distributions

R function	Density/pdf at x	Cdf at x
binom	dbinom(x, n, theta)	pbinom(x, n, theta)
pois	dpois(x, lambda)	ppois(x, lambda)
unif	dunif(x, a, b)	punif(x, a, b)
norm	dnorm(x, mu, sigma)	pnorm(x, mu, sigma)
exp	dexp(x, beta)	pexp(x, beta)
gamma	dgamma(x, alpha, beta)	pgamma(x, alpha, beta)
chisq	hisq(x, nu)	pchisq(x, nu)
t	dt(x, nu)	pt(x, nu)
weibull	dweibull(x, alpha, beta)	pweibull(x, alpha, beta)
beta	dbeta(x, alpha, beta)	pbeta(x, alpha, beta)

Table A.3 Quantiles and random number generation of built in distributions

R function	Quantile of order p	Random sample, m
binom	qbinom(p, n, theta)	rbinom(m, n, theta)
pois	qpois(p, lambda)	rpois(m, lambda)
unif	qunif(p, a, b)	runif(m, a, b)
norm	qnorm(p, mu, sigma)	rnorm(m, mu, sigma)
exp	qexp(p, beta)	rexp(m, beta)
gamma	qgamma(p, alpha, beta)	rgamma(m, alpha, beta)
chisq	qchisq(p, nu)	rchisq(m, nu)
t	qt(p, nu)	rt(m, nu)
weibull	qweibull(p, alpha, beta)	rweibull(m, alpha, beta)
beta	qbeta(p, alpha, beta)	rbeta(m, alpha, beta)

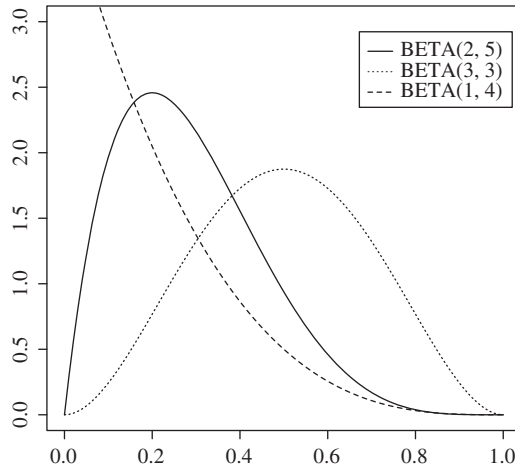


Figure A.1 The densities of BETA(2, 5), BETA(3, 3), and BETA(1, 4) distributions.

```
plot.new()
plot.window(xlim=c(0, 1), ylim=c(0, 3))
x=seq(0, 1, length=100)
y1=dbeta(x, 2, 5)
y2=dbeta(x, 3, 3)
y3=dbeta(x, 1, 4)
lines(x, y1)
lines(x, y2, lty="dotted")
lines(x, y3, lty="dashed")
axis(1)
axis(2)
box()
legend(0.7, 2.9, legend=c("BETA(2, 5)", "BETA(3, 3)",
"BETA(1, 4)"), lty=c(1, 3, 2))
```

Chapter 9.

Descriptive methods and generating random numbers. In all examples, data values are stored in vector *d* (or *d1* and *d2*). The basic statistics can be obtained using functions in Tables A.4 and A.5.

Table A.4 Basic statistics for data values stored in vector *d*

Statistic	R code
Sample mean	<code>mean(d)</code>
Sample median	<code>median(d)</code>
Standard deviation	<code>sd(d)</code>
Variance	<code>var(d)</code>
Trimmed mean	<code>mean(d, trim = p)</code> # the upper and lower 100 <i>p</i> % of the data removed
Quantile	<code>quantile(d, p)</code> # quantile of order <i>p</i>
Interquartile range	<code>quantile(d, 0.75)-quantile(d, 0.25)</code>

Table A.5 Basic graphical methods

Graph	R code
Boxplot	<code>boxplot(d)</code>
Multiple boxplots	<code>boxplot(d1, d2) #two boxplots in one plot (for d1 and d2)</code>
Dot diagram	<code>stripchart(a, method="stack", pch=1)</code>
Q-Q plot	<code>qqnorm(d) #add qqline(d) for a reference straight line</code>
Histogram	<code>hist(d, main= "Histogram for d")</code>

R code for obtaining simple graphs displaying univariate data are given in Table A.3. For a thicker line, include `lwd=2`, or `lwd=3`. The higher the value the thicker is the line. For example, compare `boxplot(d)` with `boxplot(d, lwd=3)`.

■ EXAMPLE A.2 Obtaining a Histogram with Fitted Normal Density

The code below produces a histogram with bins being relative frequencies rather than frequencies. Values of `xmin` and `xmax` should correspond to the minimum and maximum value in `d` (here 100 randomly generated observations). The density in Figure A.2 is the density of the true distribution $BETA(2, 3)$. The density in Figure A.3 is the normal density with the same mean and standard deviation as the sample mean and standard deviation in the dataset `d`.

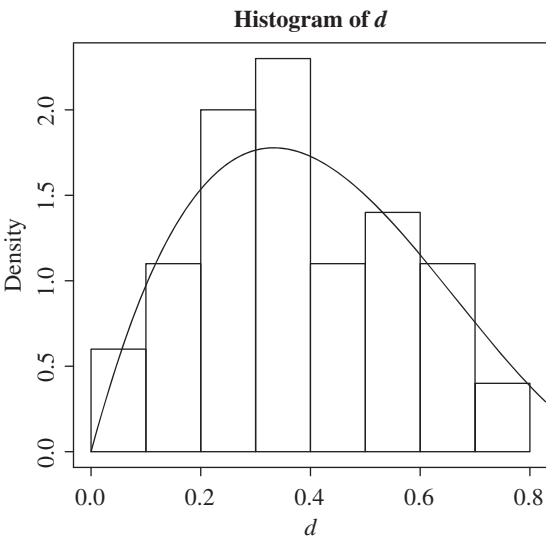


Figure A.2 Histogram of the data generated from the $BETA(2, 3)$ distribution and the $BETA(2, 3)$ density.

```
alpha<-2
beta<-3
d<-rbeta(100, alpha, beta)
plot.window(xlim=c(0, 1), ylim=c(0, 3))
hist(d, freq=FALSE)
x<-seq(0, 1, length=100)
```

```

y<-dbeta(x, alpha, beta)
lines(x, y)

or

y<-dnorm(x, mean(d), sd(d))

```

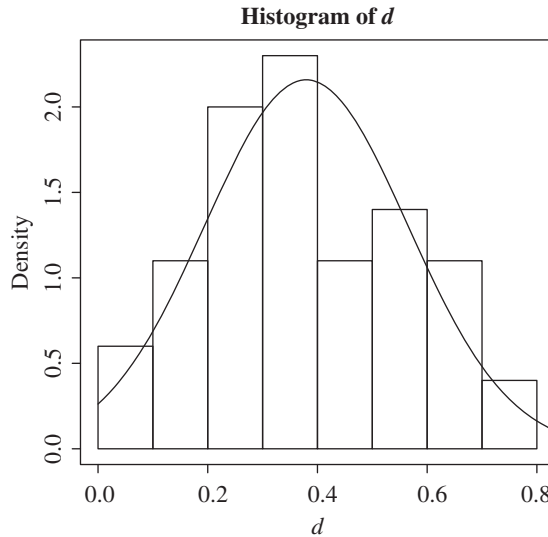


Figure A.3 Histogram of the data generated from the BETA(2, 3) distribution and the normal density with parameters same as the sample mean and variance.

Generating random data. Generating data from specific distribution requires adding `r` to the distribution's name. For example, `d<-rbeta(n, alpha, beta)` will randomly generate n observation from the BETA(alpha, beta) distribution and store them in `d`. Similar functions for generating observations from other distributions can be found in the last column of Table A.3.

Accept–reject algorithm (introduced in Section 9.4)

■ EXAMPLE A.3

We will generate 1,000 observations from the BETA(1.5, 2.5) distribution. If $g(x)$ is a $U(0, 1)$ density, then $f(x)/[cg(x)] < 1.7$ and the R code is:

```

A<- 16/(pi*1.7)
n<-1000
m<- 0           # number of obs. accepted as BETA(1.5, 2.5)
i<- 0           # number of iterations
y<-numeric(n)   # stores observations BETA(1.5, 2.5)
while (m <n) {
  u<-runif(1)
  i<-i+1
  x<-runif(1)
  if (A*x^0.5*(1-x)^1.5 > u) {
    m<-m+1
    y[m]<- x
  }
}

```

Chapter 11.

Bootstrap interval. We will be using the following notation: B —number of bootstrap samples, d —an original dataset (here of size 15), the parameter of interest in the population mean. The R-code will yield two intervals: 90% t -interval (bootl, bootu) and 90% percentile interval(percl, percu).

```
B<- 1000 # number of bootstrap repetitions, here B=1000.
y<- 1:1000
for (i in 1:B){
    resam<-sample(c, replace=T)
    y[i]<-mean(resam)
}
my<-mean(y)
se<-sqrt( mean( (y-my) ^ 2))
bootl<-my-qt(0.9, df=15)*se
bootu<-my+qt(0.9, df=15)*se
percl<- sort(y)[900]
percu<-sort(y)[101]
```

Chapter 12.

Permutation/randomization test. Permutation test is used for comparison of a certain parameter in two populations. Observations in two samples are combined and then split into two subsets of the same sizes as the original samples (m and n). All $\binom{m+n}{m}$ such different pairs of subsets should be considered. If $\binom{m+n}{m}$ is too large, subsets are selected at random a substantial number of times (here 1,000) and the test becomes a randomization test.

■ EXAMPLE A.4 The randomization test

We are testing hypothesis that the median in population 1 (x —sample of size 10) is larger than the median in population 2 (y —sample of size 15), $len = 1,000$ number of replications.

```
len<-1000
w<-c(x, y) # pooled sample
K<-1:25
reps<- numeric(len) # stores values of the statistic
for (i in 1:len) {
    k<-sample(K, size=10, replace=False)
    x1<-w[k]
    y1<-w[-k]
    reps[i]<-median(x1)-median(y1)
}
p<-mean(c(tzero, reps) >= tzero)
```